

课程报告五：基于 Unsloth 与 Qwen3 的《孙子算经》领域语言模型训练

方俊睿

学号：3230104069

仓库地址：<https://github.com/Tomabsolute/Neul.git>

1 问题背景

本次作业要求在 nanoGPT、MiniMind、Unsloth 三个框架中任选一个，完成从头训练、基于至少 Qwen3 0.6B 规模模型的微调，以及后训练实验。本文选择的数据集为《孙子算经》文本。该文本具有两个特点：第一，它是古文体数学文本，包含大量古汉语数字、度量衡单位和固定问答格式；第二，题目后常以「答曰」给出结果，因此天然适合构造「题目到答案」的指令微调数据。

本实验的目标不是训练通用大模型，而是观察一个小规模训练流程是否能使模型学习《孙子算经》的领域表达方式：能否识别「今有……问……」题式，能否在答案中稳定使用「畝、步、尺、斗、升、分之」等单位，能否倾向于输出简洁的古文答案。由于严格算术推理需要更强的符号泛化能力，本报告将「学到知识」界定为三类可观察现象：困惑度下降、生成格式接近原文、偏好训练后更少输出明显不合题意的答案。

2 框架调研与选择

2.1 三个框架的特点

nanoGPT 是 Karpathy 开源的极简 GPT 训练代码，适合理解从 tokenizer、数据 batch、Transformer 到自回归生成的完整细节。它的优势是代码短、透明、便于从头训练；不足是缺少面向现代大模型的 QLoRA、DPO、量化加载等工程封装。

MiniMind 是面向教学和复现的小型中文大模型训练框架，文档给出了 tokenizer 训练、预训练、SFT、LoRA、DPO、PPO/GRPO/SPO、蒸馏等较完整流水线。它适合系统学习「小模型从零到对齐」的全过程，尤其适合单卡复现实验。

Unsloth 的定位更偏工程化大模型微调与后训练。

官方文档说明其支持 Qwen3 系列模型的本地运行、4-bit 微调和强化学习/偏好优化，并强调训练速度和显存效率。由于本作业明确要求基于至少 Qwen3 0.6B 尺寸做微调与后训练，Unsloth 与 Qwen3、QLoRA、DPO 的结合最直接，因此本实验选择 Unsloth 作为主框架。

2.2 相关方法

本实验使用的基础模型范式是 decoder-only 自回归语言模型，其核心来自 Transformer 的自注意力结构。Transformer 使用多头注意力建模序列中任意位置之间的关系，替代循环网络中的逐步状态传递，是当前大语言模型的主流结构。

微调阶段采用 LoRA/QLoRA 思路。LoRA 冻结预训练模型主体参数，只在注意力和前馈层中注入低秩可训练矩阵，从而显著减少可训练参数量。QLoRA 进一步把基础模型以 4-bit 方式量化加载，并只训练 LoRA adapter，适合在单张消费级 GPU 上微调较大的模型。

后训练阶段采用 DPO。DPO 将偏好数据表示为 chosen/rejected 回复对，直接优化模型更偏好 chosen 回复，而不需要显式训练 reward model，也不需要 PPO 式在线采样。这与本实验中「正确古文答案」优于「单位或数字被扰动的答案」的偏好构造方式匹配。

3 数据集构造

原始数据使用 Project Gutenberg 提供的《孙子算经》UTF-8 文本，默认保存为 `work5/data/sunzi_suanjing.txt`。脚本 `work5/scripts/prepare_sunzi_dataset.py` 会在文件不存在时自动下载，并尝试 UTF-8、GB2312、GB18030、GBK 编码读取，随后抽取包含「答曰」的题答对。

输出数据包括三种格式：

- `pretrain.txt`：完整规范化文本，用于从头训练自回归语言模型；

- `sft.jsonl`: messages 格式，包含 system、user、assistant 三段，用于 Qwen3 指令微调；
- `dpo.jsonl`: prompt、chosen、rejected 三列，其中 rejected 由扰动答案中的数字或单位生成，用于偏好后训练。

表 1: 数据构造样例

字段	内容示例
user	今有物不知其數，三三數之剩二，五五數之剩三，七七數之剩二。問物幾何？
assistant	二十三。
rejected	二十二。或二十四。

4 算例一：从头训练

从头训练部分使用随机初始化的小型 decoder-only Transformer 作为可控基线。该模型不是为了超过 Qwen3，而是为了验证同一份古文数学语料能否驱动自回归语言模型学习局部统计规律。模型按字符建模，避免古文分词带来的额外误差。

表 2: 从头训练模型配置

项目	配置
Token 粒度	字符
层数	6
注意力头数	8
隐藏维度	384
上下文长度	256
优化器	AdamW
学习率	3×10^{-4}

服务器运行后生成
`results/scratch/curves.png`

图 1: 从头训练损失曲线

5 算例二：Qwen3 0.6B 微调

微调阶段使用 `unsloth/Qwen3-0.6B-unsloth-bnb-4bit`，Qwen3 技术报告中说明 Qwen3 系列覆盖 0.6B 到 235B 参数规模，并包含 dense 与 MoE 架构；0.6B 模型适合课程作业中的单卡实验。Unsloth 文档也明确给出 Qwen3 0.6B 的 4-bit 微调入口。

表 3: Qwen3 SFT 配置

项目	配置
基础模型	Qwen3 0.6B 4-bit
训练方法	LoRA / QLoRA SFT
LoRA rank	16
LoRA alpha	32
目标模块	q/k/v/o 与 MLP 投影层
最大长度	1024
学习率	2×10^{-4}

微调输入采用 Qwen chat template，将题目放在 user 消息中，将「答曰」后的文本作为 assistant 回复。预期微调后的模型在面对类似「今有……问……」的题目时，会更倾向于直接输出古文答案，而不是现代解释性长段落。

6 算例三：偏好后训练

后训练阶段使用 DPO。对每个题目构造一条 chosen 答案和一条 rejected 答案。chosen 使用原文答案，rejected 通过替换数字或单位得到，例如将「二十三」扰动为「二十二」或将「尺」扰动为「寸」。这种自动构造并不等同于完整人工偏好标注，但足以表达本实验中的偏好：模型应优先输出符合原文答案格式、数字和单位的回复。

表 4: DPO 后训练配置

项目	配置
初始化	Qwen3 SFT adapter
偏好算法	DPO
偏好系数 β	0.1
学习率	5×10^{-5}
Batch size	1
梯度累积	8

7 运行方式

完整脚本位于 `work5/`。服务器可执行：

```
pip install -r work5/requirements.txt
bash work5/run_all_experiments.sh
```

其中 `SCRATCH_STEPS`、`SFT_STEPS`、`DPO_STEPS` 可通过环境变量调整。若显存不足，可降低 batch size 或 max

sequence length; 若训练时间充足,可增加 SFT 与 DPO step 数。

8 结果记录表

由于 Qwen3 0.6B 微调和 DPO 需要服务器 GPU 与模型下载,本报告保留结果记录表。服务器运行后将 `results/*/metrics.json` 中的实际数值填入即可。

表 5: 实验结果记录

阶段	指标	数值
数据准备	题答对数量	65
从头训练	最终验证 loss / PPL	运行后填入
Qwen3 SFT	train loss	运行后填入
Qwen3 DPO	train loss	运行后填入
生成测试	物不知数题输出	运行后填入

推荐的人工评测题包括: 物不知数题、鸡兔同笼题、度量衡题、余数同余题和分配题。评测时重点观察三点: 是否输出简短答案, 是否保留古文数字和单位, 是否避免把其他题型的答案迁移到当前题目。

9 分析与不足

从头训练的小模型可以观察到语言建模损失下降, 但由于《孙子算经》语料规模有限, 随机初始化模型更容易学习局部字词搭配, 而不具备稳定解题能力。Qwen3 SFT 借助预训练模型已有的中文和数学能力, 预计能更快学习任务格式。DPO 后训练进一步把「原文答案」和「扰动答案」拉开, 有助于减少单位错配和明显数值错误。

本实验的主要限制是偏好数据为自动扰动生成, 质量低于人工标注; 此外,《孙子算经》原文题目数量有限, 训练集与测试集之间存在题型重复, 生成正确并不必然说明模型学会了真正的算术推理。后续可构造保持题型但替换数字的合成测试集, 单独评价模型的符号计算泛化能力。

10 结论

本作业选择 Unsloth 作为主框架, 围绕《孙子算经》构造了从原始文本到预训练、SFT、DPO 的完整实验流程。代码覆盖从头训练基线、Qwen3 0.6B QLoRA 微调和偏好后训练, 并提供可在服务器上复现实验的脚本。该流程能够展示模型是否学习到古文数学文本中的

格式、术语和单位知识, 同时也明确区分了文本模式学习与严格算术推理之间的差异。

参考文献

[1] Vaswani et al. Attention Is All You Need. arXiv:1706.03762, 2017. <https://arxiv.org/abs/1706.03762>

[2] Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685, 2021. <https://arxiv.org/abs/2106.09685>

[3] Dettmers et al. QLoRA: Efficient Finetuning of Quantized LLMs. arXiv:2305.14314, 2023. <https://arxiv.org/abs/2305.14314>

[4] Rafailov et al. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. arXiv:2305.18290, 2023. <https://arxiv.org/abs/2305.18290>

[5] Qwen Team. Qwen3 Technical Report. arXiv:2505.09388, 2025. <https://arxiv.org/abs/2505.09388>

[6] Unsloth Documentation. Qwen3: How to Run and Fine-tune. <https://unsloth.ai/docs/models/qwen3-how-to-run-and-fine-tune>

[7] MiniMind Documentation. Model Training Guide. <https://minimind.readthedocs.io/en/stable/training/>

[8] Karpathy. nanoGPT: The simplest, fastest repository for training/finetuning medium-sized GPTs. <https://github.com/karpathy/nanoGPT>

[9] Project Gutenberg. Sunzi Suanjing. <https://www.gutenberg.org/ebooks/24038>